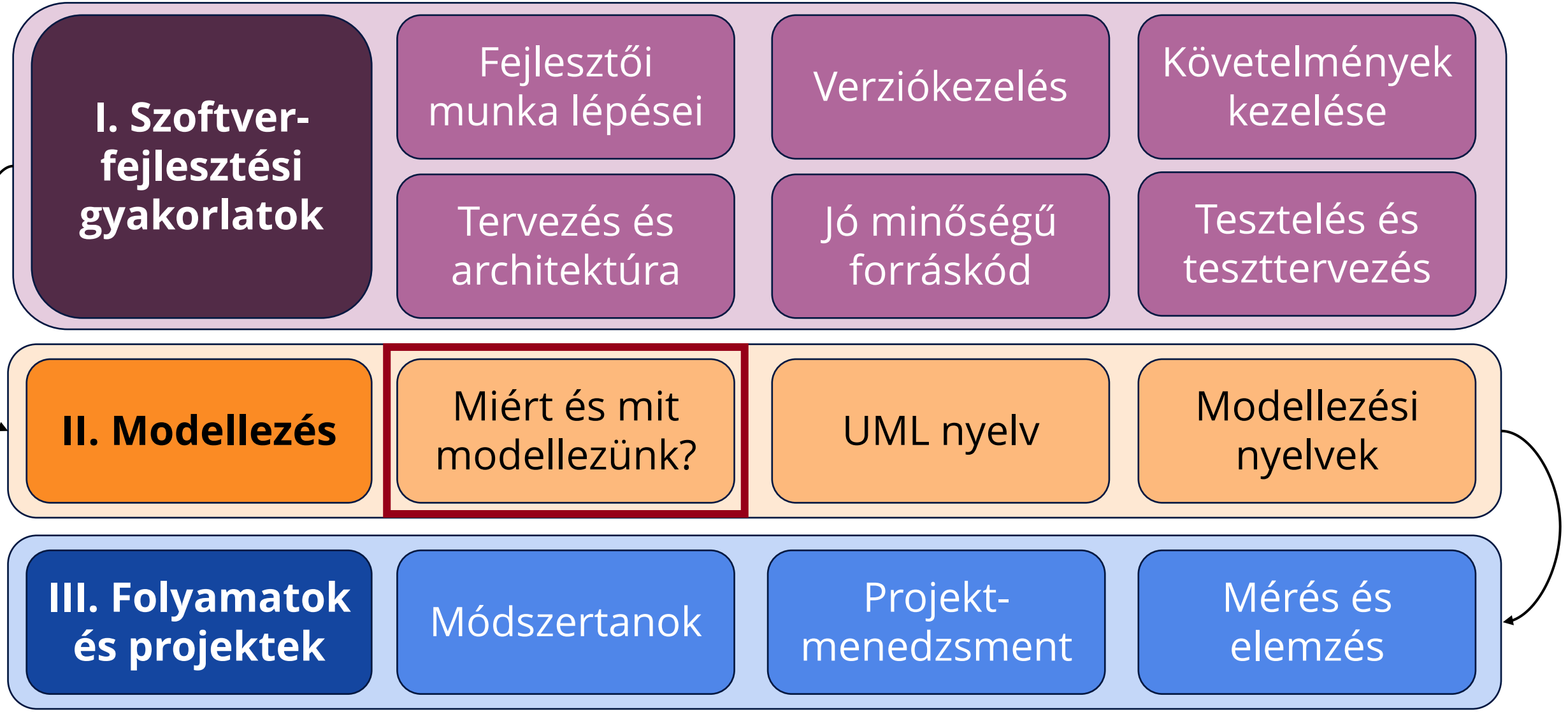
Szoftvermodellezés

Dr. Micskei Zoltán, Dr. Gönczy László
https://ftsrg.mit.bme.hu

MŰEGYETEM 1782
BME VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
Mesterséges Intelligencia és Rendszertervezés Tanszék
ftsrg Critical Systems Research Group

Szoftvertechnológia tantárgy témái



Tanulási eredmények

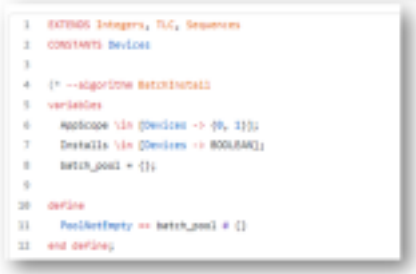
A tananyag elsajátítása után a hallgató képes lesz

- (K2) összefoglalni a modellezés célját és jellegzetességeit,
- (K1) felsorolni informatikai modellezési nyelveket.

Modellezési alapok

Miért? Hogyan? Mikor?









Milyen modellezési nyelveket ismertek vagy használtatok?

Mi a modell?

- Egy valós vagy hipotetikus világ (a „*rendszer*”)
  - egy **részének**
  - **egyszerűsített képe**, amely
  - a rendszert **helyettesíti** bizonyos megfontolásokban
- Döntések:
  - **A világ mely része**?
  - **Mit hanyagol el**?
  - **Hogy feleltethető meg a világnak**?
- Haszna
  - kisebb (véges)
  - áttekinthetőbb

Mikor *lehet* és *érdemes* felhasználni?

Tudományos és mérnöki modellek

Tudományos modell
deskriptív (leíró)

- Valóság („rendszer”) megértésére készítjük
- Érték: mennyire jól írja le a modellezett rendszert
- Newton törvénye nem illeszkedik fekete lyukra

  → rossz modell

Mérnöki (tervezési) modell
preskriptív (előíró)

- Tervként, specifikációként szolgál egy rendszerhez
- Érték: az elkészített rendszer mennyire jól illik a modellhez
- Nem illeszkedik az elkészült NYÁK a modellre/tervre

  → „rendszer" rossz

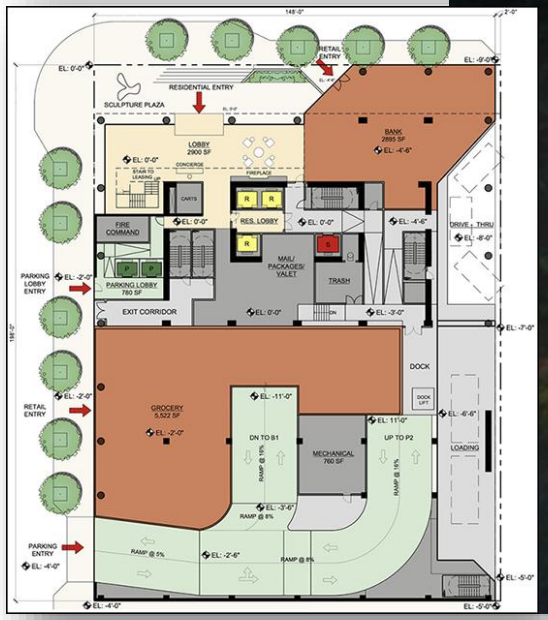
E. A. Lee. Modeling in engineering and science. DOI

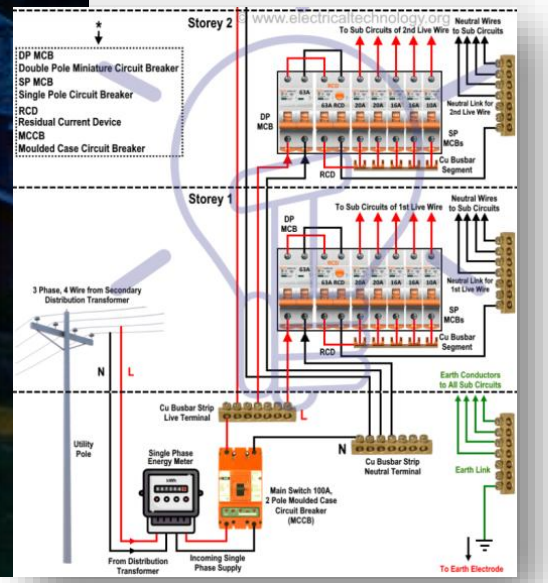
Mérnöki modellek készítése és felhasználása











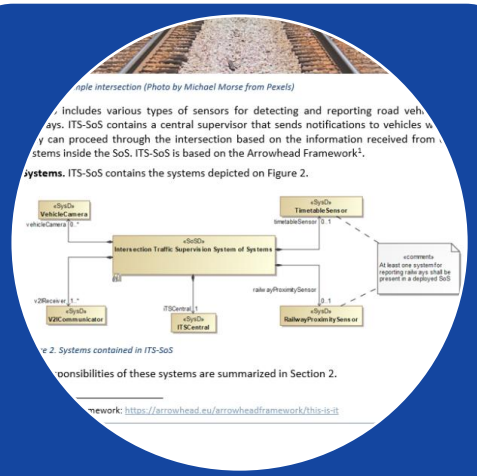
Mi értelme van modellezni?

- Én szoftvert készítek. Kell-e modelleznem is?
  - Már így is ezt teszed!
    - (A szoftver forráskódja is egyfajta modell…)
    - Ami fontosabb: **mentális modellek**
- Mikor kell kifejezetten *lejegyezni* a modelleket?
  - Szerepe: **kommunikáció**
    - Ember → ember
    - Ember → gép
    - Gép → gép
    - Ember → saját maga, kicsivel később
      - Pl. évekkel később emlékezni kéne mérnöki döntések indokaira…

Modellezési stílusok



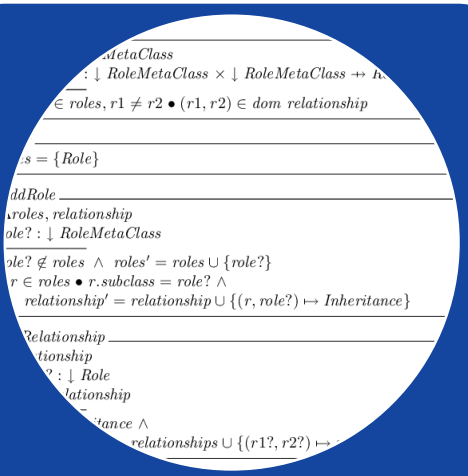
Brainstorming



Illusztráció



Részletes specifikáció



Formális modell

Egyre precízebb modellek és modellezési nyelvek

Alapfogalmak: Finomítás és Absztrakció

- *Finomítás*: a modell gazdagítása részletekkel…

  Pl. rendszer felbontása alrendszerekre

  …hogy az eredeti modell absztrakció maradjon

  Pl. az alrendszerekre együtt ugyanazokat az elvárásokat teljesítik

- Inverze: *absztrakció*

  Pl. alrendszerek összevonása

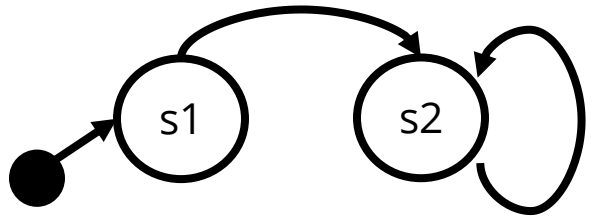
- Finomítás fajtái, pl.
  - Hierarchikus finomítás: „dobozok kibontása”
  - Halmazfinomítás: változók értékkészletének finomítása
    - **Jó** / **rossz** helyett **Gyors** / **átlagos** / **lassú** / **hiányos** / **veszélyes**

Modellezési nyelv

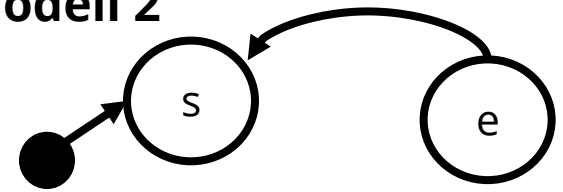
- Megadja, hogy mik a jól formált [well-formed] modellek

- Nyelv definiálásához szükséges:
  - **Szintaxis**: milyen szabályok alapján lehet érvényes modellt építeni?
  - **Szemantika**: mit jelent a modell? Milyen tartományba képezhetjük le?

Modell 1



Modell 2

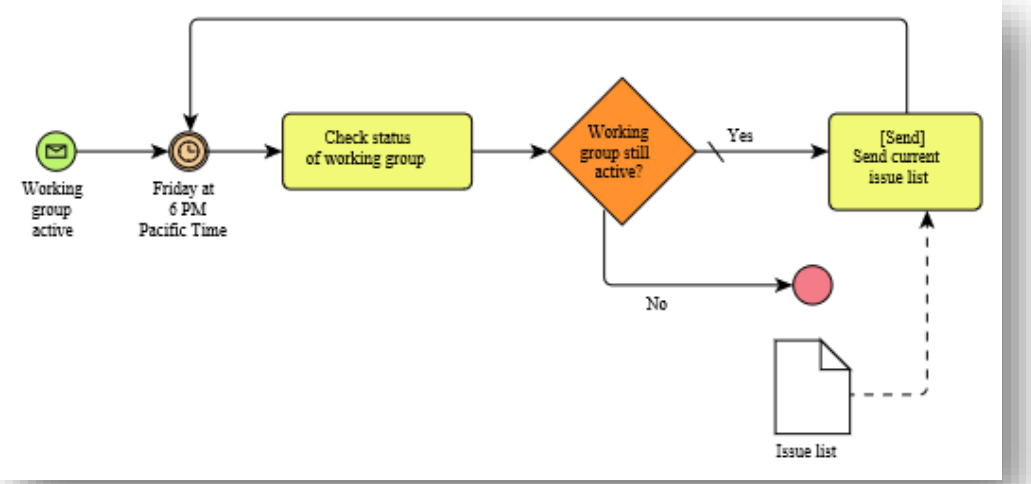


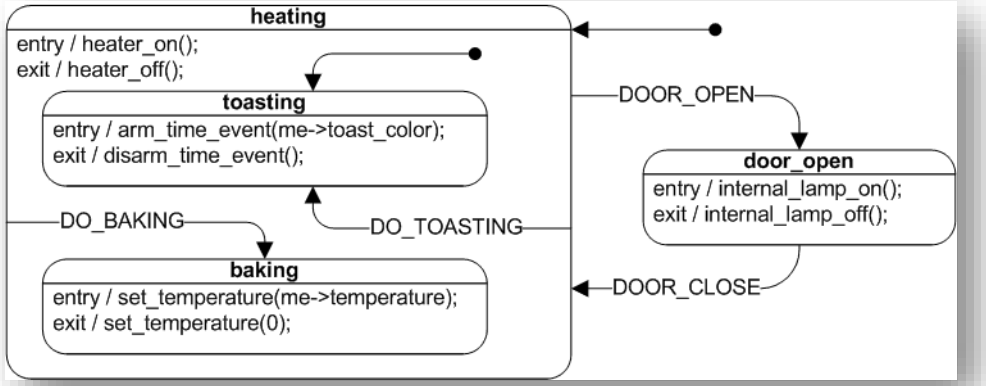
Modellezési nyelv

Szintaxis
- Átmenet (nyíl) forrás- és célállapotot (kör) köt össze.
- Forrás- és célállapot lehet ugyanaz

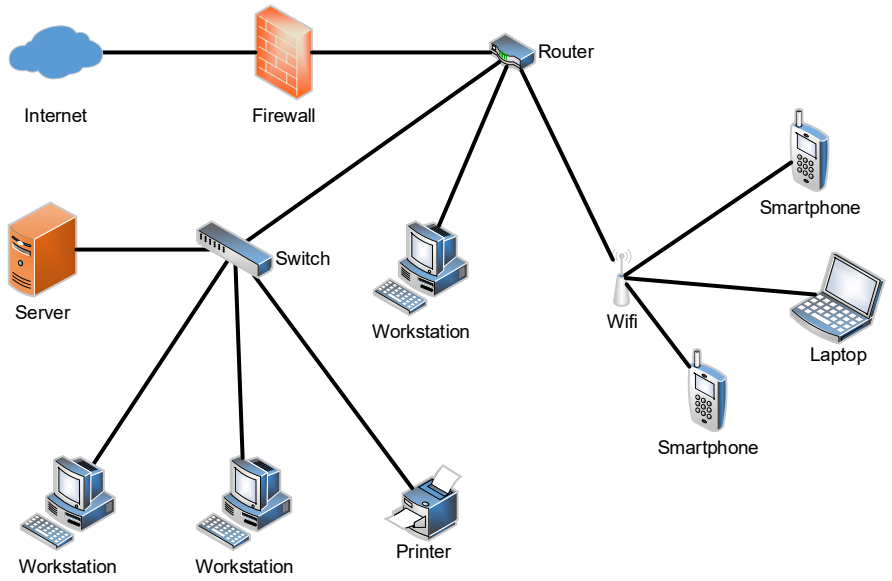
Szemantika
- Átmenetek eseményekre adott állapotváltozást mutatják be
- Modell jelentése állapotváltozások sorozata

Tipikus informatikai modellek és nyelvek






```
1   EXTENDS Integers, TLC, Sequences
2   CONSTANTS Devices
3
4   (* --algorithm BatchInstall
5   variables
6     AppScope \in [Devices -> {0, 1}];
7     Installs \in [Devices -> BOOLEAN];
8     batch_pool = {};
9
10  define
11    PoolNotEmpty == batch_pool # {}
12  end define;
```



Szöveges és grafikus nyelveket is használunk!

Összefoglalás

Mi a modell?

- Egy valós vagy hipotetikus világ (a „*rendszer*”)
  - egy **részének**
  - **egyszerűsített képe**, amely
  - a rendszert **helyettesíti** bizonyos megfontolásokban
- Döntések:
  - **A világ mely része**?
  - **Mit hanyagol el**?
  - **Hogy feleltethető meg a világnak**?

Mikor *lehet* és *érdemes* felhasználni?

- Haszna
  - kisebb (véges)
  - áttekinthetőbb

ftsrg

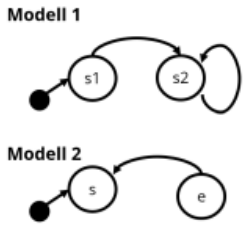
Modellezési stílusok



ftsrg

Modellezési nyelv

- Megadja, hogy mik a jól formált [well-formed] modellek
- Nyelv definiálásához szükséges:
  - **Szintaxis**: milyen szabályok alapján lehet érvényes modellt építeni?
  - **Szemantika**: mit jelent a modell? Milyen tartományba képezhetjük le?



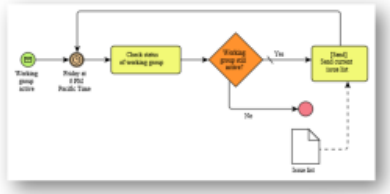
Modellezési nyelv

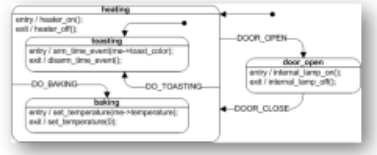
Szintaxis
- Átmenet (nyíl) forrás- és célállapotot (kör) köt össze.
- Forrás- és célállapot lehet ugyanaz

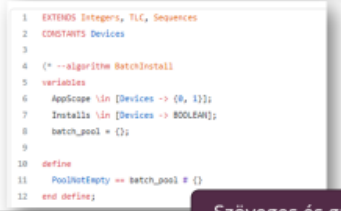
Szemantika
- Átmenetek eseményekre adott állapotváltozást mutatják be
- Modell jelentése állapotváltozások sorozata

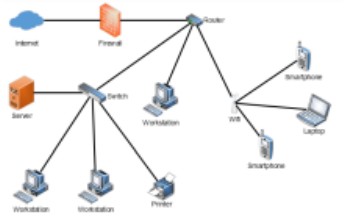
ftsrg

Tipikus informatikai modellek és nyelvek







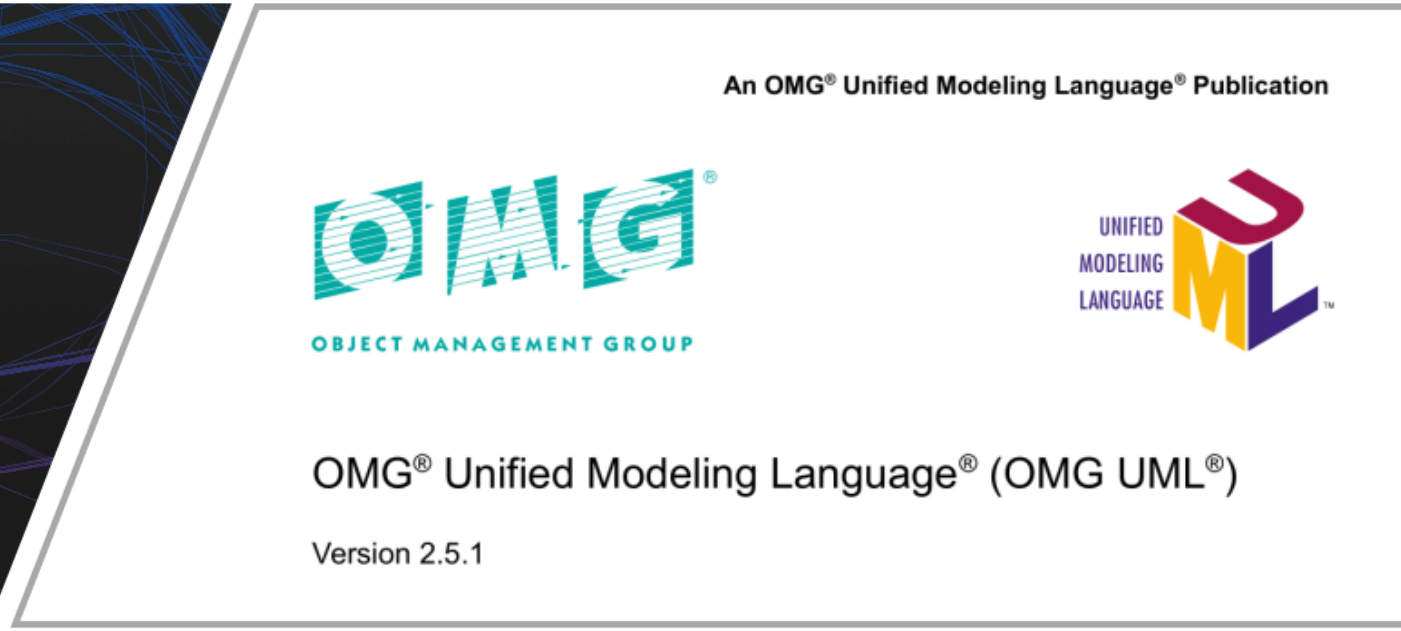


Szöveges és grafikus nyelveket is használunk!

ftsrg

ftsrg

Unified Modeling Language (UML)

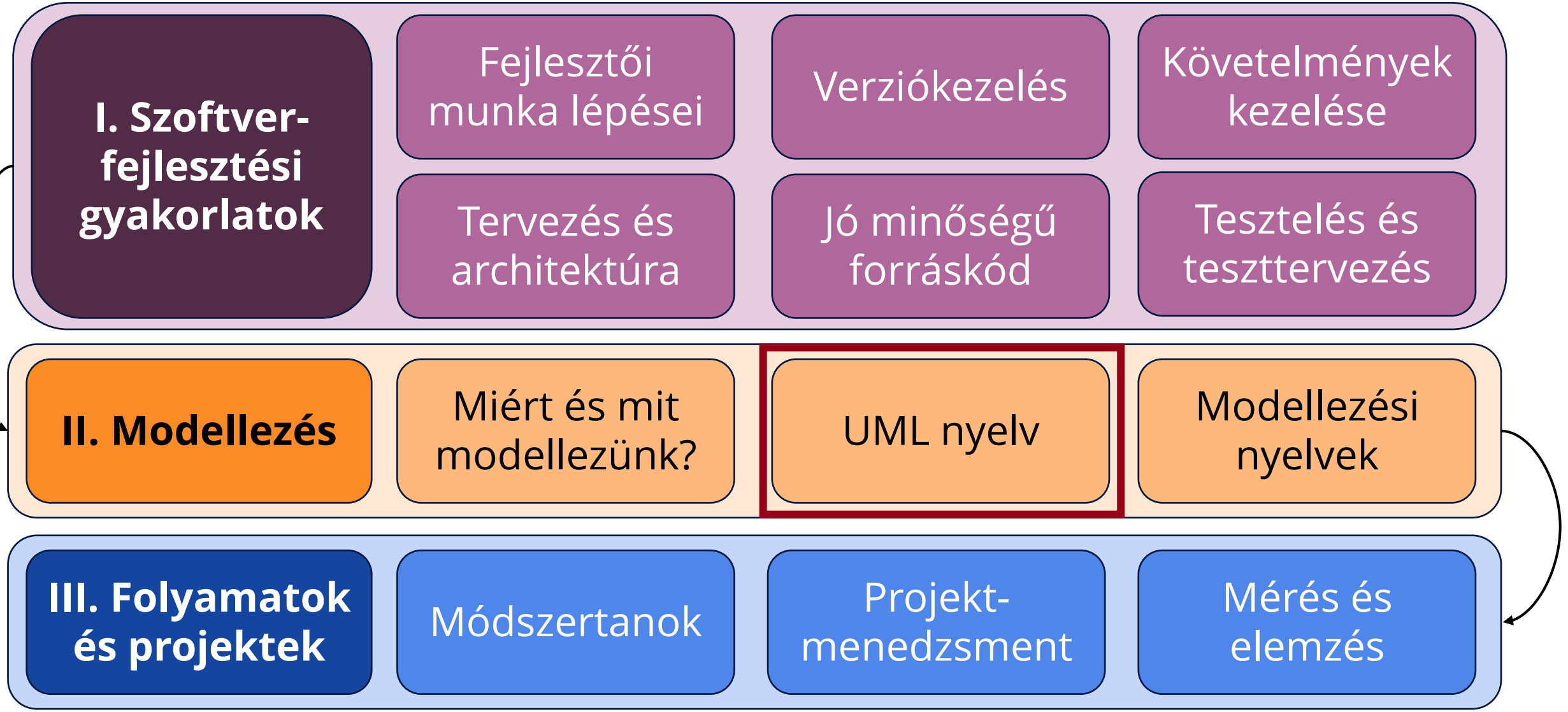


Tanulási eredmények

A tananyag elsajátítása után a hallgató képes lesz

- (K1) azonosítani az UML modellezési nyelv jellemzőit.

Szoftvertechnológia tantárgy témái



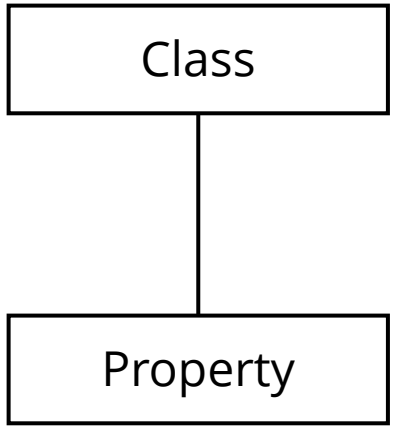
Unified Modeling Language (UML)

- Szabványos nyelv szoftverrendszerek leírására
  - Vizuális, általános célú, közös nyelv
- Object Management Group (OMG) gondozza
- Történet
  - 90’-es évek: sokféle jelölés és módszertan
  - 1997: UML 1.0 javaslat (G. Booch, I. Jacobsen, J. Rumbaugh és mások)
  - 2005: UML 2.0, jelentősen megújított, közös szemantikai alapok
  - 2017: UML 2.5, egyszerűsített leírás, egy dokumentum
  - 2011- precíz szemantika leírása (fUML, PSCS, PSSM…)

Modell és diagram

Egy diagramon a modell elemeinek egy része jelenik meg

UML nyelv



Modell

```
Person [Class]
  Name [Property]
  Age [Property]

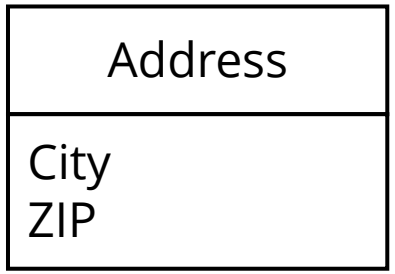
Address [Class]
  City [Property]
  ZIP [Property]

Country [Class]
  Capital [Property]
```

Diagram 1

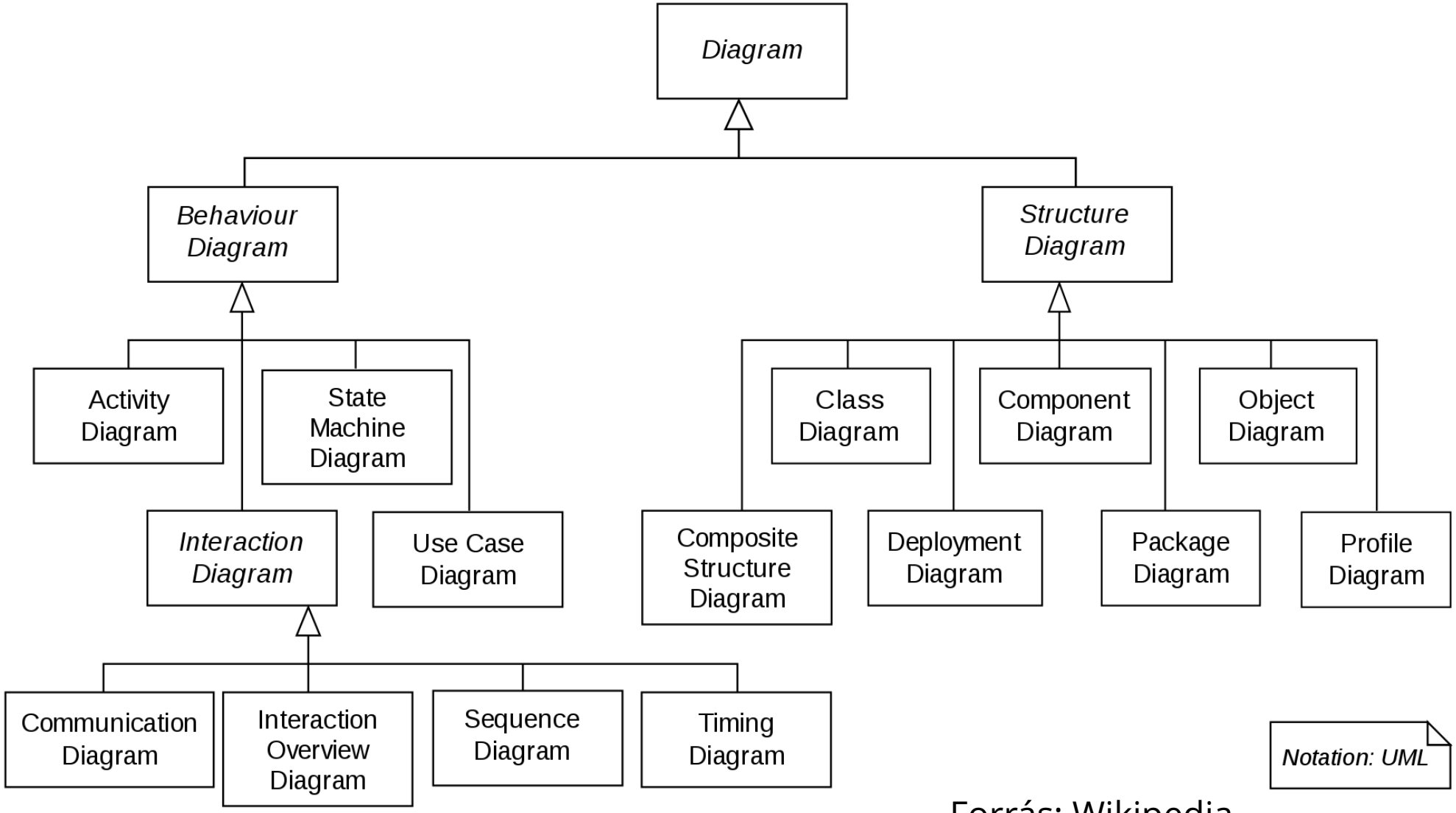


Diagram 2



modell ≠ diagram

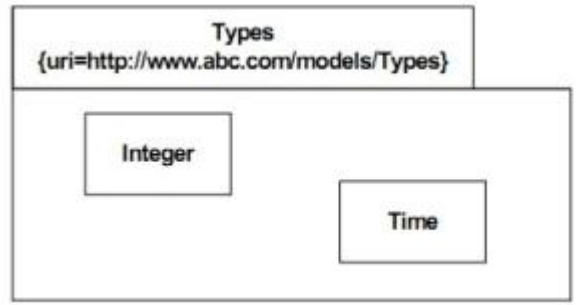
UML diagram típusok

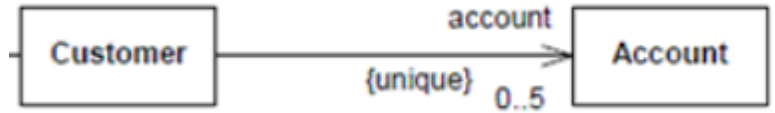


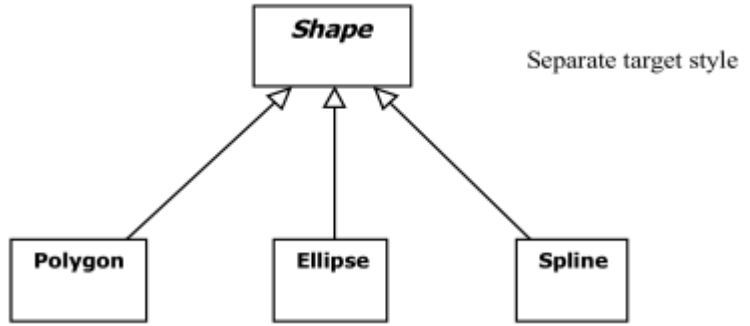
Forrás: Wikipedia

UML fogalmak és alapelemek

- Element, Relationship: legáltalánosabb, absztrakt elemek
- Package [csomag]: szervezésre, modellelemek csoportosítására
- MultiplicityElement: olyan elem, amiből több példány lehet (alsó és felső korlát)
- Classifier: példányokat osztályba sorol, lehetnek úgynevezett Feature-jei
- Generalization: irányított kapcsolat, ami hierarchiába szervezi a többieket



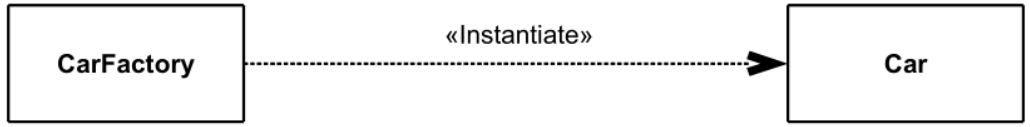


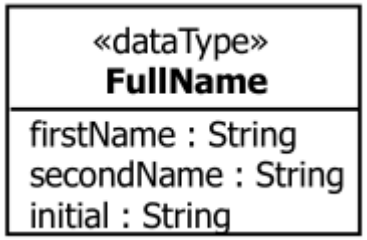


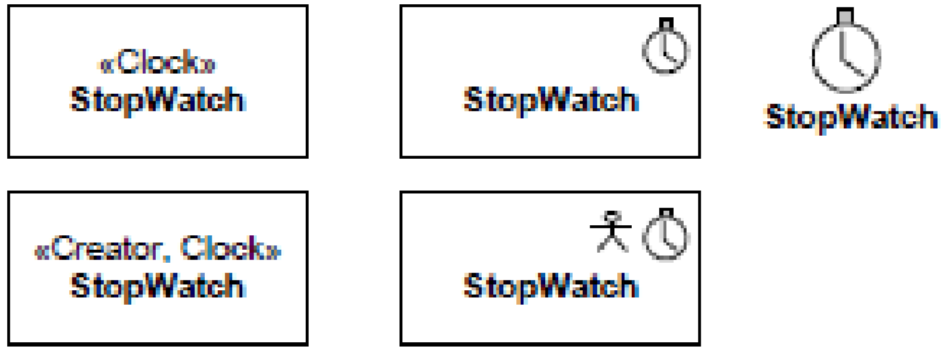
UML sztereotípia

- UML modellezési elemek kiterjesztése
  - Beépített UML sztereotípiák (pl. component, interface)
  - Saját kiegészítés egy profil [profile] részeként
- Jelölés: « » jelek (guillemets) közötti név

Példák:







Az UML specifikáció

https://www.omg.org/spec/UML

9 Classification

9.1 Summary

Classification is an important technique for organization. This clause specifies concepts relating to classification. The core concept is Classifier, an abstract metaclass whose concrete subclasses are used to classify different kinds of values. The other metaclasses in this clause represent the constituents of Classifiers, models of how Classifiers are instantiated using InstanceSpecifications, and various relationships between all of these concepts.

9.2 Classifiers

9.2.1 Summary

A Classifier represents a classification of instances according to their Features. Classifiers are organized in hierarchies by Generalizations. RedefinableElements may be redefined in the context of Generalization hierarchies.

9.2.2 Abstract Syntax

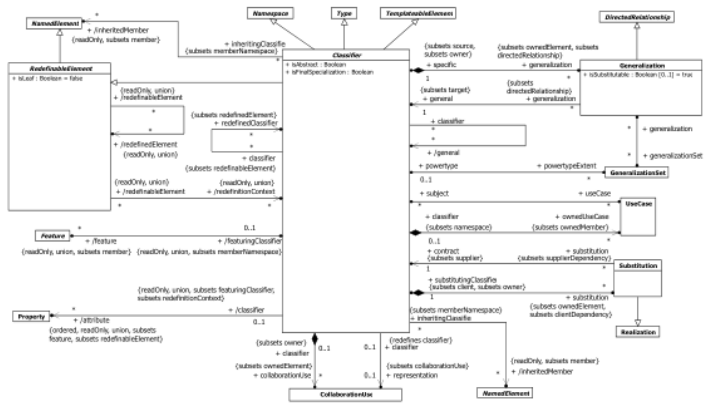


Figure 9.1 Classifiers

9.2.3 Semantics

9.2.3.1 Classifiers

A Classifier has a set of [illegible] Properties called the attributes of the Classifier. Each of the Features is a [illegible]

The [illegible]

A Cl[illegible]

Unif[illegible] 99

Elemkészlet áttekintése (absztrakt szintaxis)

A Classifier may own CollaborationUses that relate the Classifier to Collaborations. The Collaborations describes aspects of this Classifier. See 11.7 Collaborations.

A Classifier may own UseCases. See 18.1 Use Cases.

9.2.3.2 Generalization

Generalizations define generalization/specialization relationships between Classifiers. Each Generalization relates a specific Classifier to a more general Classifier. Given a Classifier, the transitive closure of its general Classifiers is often called its *generalizations*, and the transitive closure of its specific Classifiers is called its *specializations*. The immediate generalizations are also called the Classifier's parents, and where the Classifier is a Class, its superClasses (see 11.4).

NOTE. The concept of parent (a generalization relationship between Classifiers) is unrelated to the concept of owner (a composition relationship between instances).

An instance of a Classifier is also an (indirect) instance of each of its generalizations. Any Constraints applying to instances of the generalizations also apply to instances of the Classifier.

When a Classifier is generalized, certain members of its generalizations are *inherited*, that is they behave as though they were defined in the inheriting Classifier itself. For example, an inherited member that is an attribute may have a value or collection of values in any instance of the inheriting Classifier, and an inherited member that is an Operation may be invoked on an instance of the inheriting Classifier.

The set of members that are inherited is called the inheritedMembers. Unless specified differently for a particular kind of Classifier, the inheritedMembers are members that do not have private visibility.

Type conformance means that if one Type conforms to another, then any instance of the first Type may be used as the value of a TypedElement whose type is declared to be the second Type. A Classifier is a Type, and conforms to itself and to all of its generalizations.

The isAbstract property of Classifier, when true, specifies that the Classifier is abstract, i.e., has no direct instances: every instance of the abstract Classifier shall be an instance of one of its specializations.

If one Classifier (the parent) generalizes another (the child) it is not necessarily the case that instances of the child are substitutable for instances of the parent under every possible circumstance. For example, Circle may be defined as a specialization of Ellipse, and its instances would be substitutable in every circumstance involving accessing the properties of an Ellipse. However, if Ellipse were to define a stretch behavior that modifies the length of its major axis only, then a Circle object would be unable to implement such a behavior. The isSubstitutable property may be used to indicate whether the specific Classifier can be used in every circumstance that the general Classifier can be used.

9.2.3.3 Redefinition

Any member (that is a kind of RedefinableElement) of a generalization of a specializing Classifier may be redefined instead of being inherited. Redefinition is done in order to augment, constrain, or override the redefined member(s) in the context of instances of the specializing Classifier. When this occurs, the redefining member contributes to the structure or behavior of the specializing Classifier in place of the redefined member(s); specifically, any reference to a redefined member in the context of an instance of the specializing Classifier shall resolve to the redefining member (note that to avoid circularity "any reference" here excludes the redefinedElement reference itself).

The Classifier from which the member [illegible] redefined is called the redefinitionContext. Although in the metamodel redefinitionContext has the multiplicity [illegible] no cases in the UML specification where there is more than one redefinitionContext. The redefinitionC[illegible] for each kind of RedefinableElement; it is often, but not always, the owner of the member.

A redefining element shall[illegible] finableElement it redefines, but it may add specific constraints or other det[illegible] general [illegible]

One red[illegible] redefin[illegible]

100

Szemantika leírása (természetes nyelven)

conflicts may occur. Instances that invoke a BehavioralFeature need to coordinate so that only one invocation to a target on any BehavioralFeature occurs at once.

- guarded
  Multiple invocations of a BehavioralFeature that overlap in time may occur to one instance, but only one is allowed to commence. The others are blocked until the performance of the currently executing BehavioralFeature is complete. It is the responsibility of the system designer to ensure that deadlocks do not occur due to simultaneous blocking.

- concurrent
  Multiple invocations of a BehavioralFeature that overlap in time may occur to one instance and all of them may proceed concurrently.

9.9.4 Classifier [Abstract Class]

9.9.4.1 Description

A Classifier represents a classification of instances according to their Features.

9.9.4.2 Diagrams

Classifiers, Classifier Templates, Features, Instances, Generalization Sets, Executable Nodes, Use Cases, Structured Classifiers, Classes, Associations, Components, Collaborations, State Machine Redefinition, DataTypes, Signals, Interfaces, Information Flows, Artifacts, Actions, Accept Event Actions, Object Actions

9.9.4.3 Generalizations

Namespace, Type, TemplateableElement, RedefinableElement

9.9.4.4 Specializations

Association, StructuredClassifier, BehavioredClassifier, DataType, Interface, Signal, InformationItem, Artifact

9.9.4.5 Attributes

- isAbstract : Boolean [1..1] = false
  If true, the Classifier can only be instantiated by instantiating one of its specializations. An abstract Classifier is intended to be used by other Classifiers e.g., as the target of Associations or Generalizations.

- isFinalSpecialization : Boolean [1..1] = false
  If true, the Classifier cannot be specialized.

9.9.4.6 Association Ends

- /attribute : Property [0..*]{ordered, union, subsets Classifier::feature, subsets A_redefinitionContext_redefinable[illegible]ment::redefinableElement} (opposite A_attribute_classifier::classifier)
  All of the Properties that are di[illegible] not inherited or imported) attributes of the Classifier.

- ♦ collaborationUse : Coll[illegible] {subsets Element::ownedElement} (opposite A_collaborationUse_c[illegible]
  The CollaborationU[illegible]

132

Elemkészlet részletei („metamodell")

UML és fejlesztési módszertanok

- UML nyelv nem rögzít fejlesztési módszertant

- Használható sok környezetben és módszerrel
  - agilis,
  - Unified process,
  - V-modell,
  - …

- Hosszú szöveges specifikációt kiválthat egy (grafikus) modell